The Electric Board Map Layout Algorithm

Octilinear routing, squeeze compaction and nestled forest packing

Technical description of the implementation in `electric_map.html`
(a single-page DHTML application)

Abstract

Korst, Pronk and van Wijk's *family metro map* draws family relations in the octilinear style of the London Underground, with segment orientation encoding gender: orthogonal for women, diagonal for men [1]. The *Electric Board Map* carries this style and gender rule over to a printed circuit board: vertices of a forest become *pads* on grid points, edges become *traces* of unit octilinear steps, and the direction of the last step into a vertex encodes its gender. The algorithm targets forests made of *large* trees (≥ 9 vertices) and many *small* ones (≤ 3). Large trees are routed generation by generation with straight, one-bend, dogleg and chamfered-corner traces, a cost that rewards parallel "bus" runs, exact branch-and-bound pruning, and a squeeze pass that rips up and reroutes the board's sides. Small trees are never routed: they are drawn from straight-only templates (a two-vertex tree is always one straight trace) and nestled into the gaps of the large trees on a flat board grid. We define the model, prove its key properties, give all constants, and report measurements.

Contents

1 Introduction **1**

2 Drawing model **2**

3 Input and validation **4**

4 Routing a large tree **4**

5 Forest packing **10**

6 Verification **12**

7 Running time **13**

8 Results **14**

9 Discussion and limitations **16**

A Parameters **17**

B Mapping to the implementation **18**

1 Introduction

The family metro map [1] draws a family as a metro line whose end stations are the parents and whose intermediate stations are the children; a child that founds a family of its own starts a side line through a T-junction. All segments are horizontal, vertical or diagonal, consecutive stations on a line are equally spaced, and the orientation of a station's segment encodes gender: women sit on horizontal or vertical segments, men on diagonal ones (inspired by the Venus and Mars symbols). The result is a compact, attractive overview that uses space more efficiently than a layered pedigree chart.

The Electric Board Map keeps three ingredients of that design:

1. an **octilinear** drawing style (all segments at multiples of 45°),

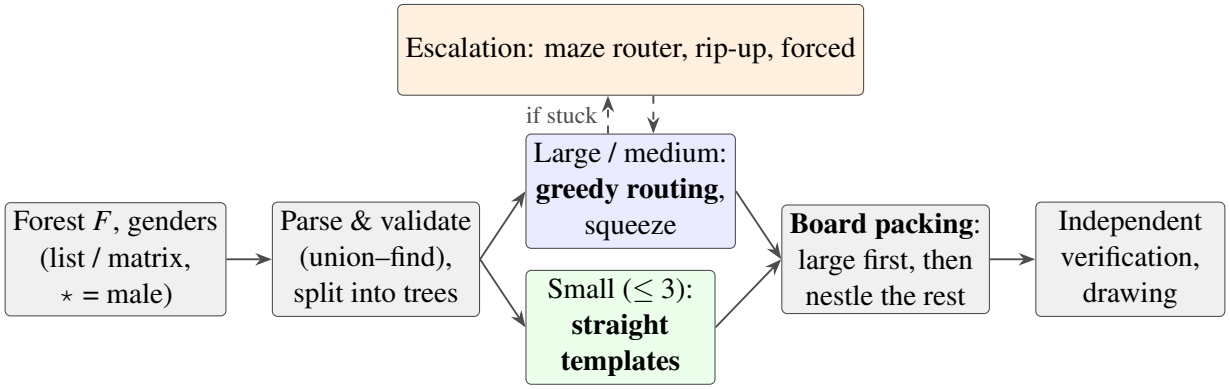


Figure 1: Pipeline. Large and medium trees are routed (blue); the escalation steps (orange) are used only when no simple trace fits. Small trees skip the router (green). Every edit of the graph re-runs the whole pipeline.

2. the **gender rule**, *orthogonal = female, diagonal = male*, and
3. a **generation-by-generation** construction from a root,

but replaces the metro-line model by a circuit-board model:

- the input is any *forest* (several trees, isolated vertices allowed), not only binary family trees;
- every edge is drawn individually as a *trace* on an integer grid, so all coordinates are exact;
- segments of a trace may have *unequal* lengths: straight, one 45° bend, a dogleg, or a chamfered 90° corner (the way circuit boards turn a corner);
- traces may run on *adjacent* grid lines, and the cost rewards such parallel runs ("buses");
- the aim is *compactness* (small board, short traces) rather than equal spacing.

Target forests. The implementation is optimised for one kind of input: forests that consist of *large* trees (≥ 9 vertices) and *small* trees (≤ 3 vertices, i.e. isolated vertices, single edges and paths of length two); *medium* trees (4–8 vertices) are rare. Large and medium trees are routed (Section 4); small trees are drawn from straight-only templates and nestled into or next to the large trees (Section 5). A tree of two vertices is always drawn as one straight trace without bends.

Figure 1 shows the overall pipeline; Figure 12 shows a typical result.

Outline. Section 2 defines the drawing model and its constraints and proves two basic properties. Section 3 covers input and validation. Section 4 describes the routing of a large tree: root and order, candidate traces, legality and reservation, the cost function, escalation, rip-up, multi-start and the squeeze pass. Section 5 describes the small-tree templates and the packing of the forest, Section 6 the verification, Section 7 running time and the exact speed-ups, and Section 8 measurements. Section 9 discusses limitations.

2 Drawing model

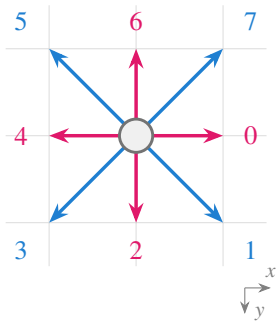
2.1 Input

The input is a forest $F = (V, E)$ with $|V| \leq 300$ and a gender function $g : V \to \{\mathrm{M}, \mathrm{F}\}$. Each connected component is a tree. For a tree T with a chosen root r, every non-root vertex v has a parent $\mathrm{par}(v)$; $\mathrm{ch}(v)$ is the list of children, $\kappa(v) = |\mathrm{ch}(v)|$, and $s(v)$ is the number of vertices of the sub-tree rooted at v.

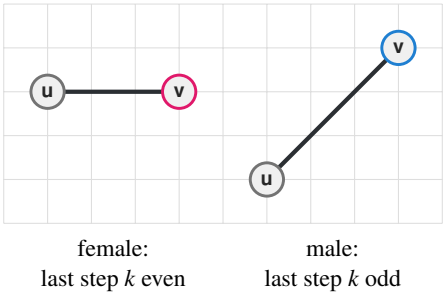
2.2 Grid, directions and traces

Drawings live on the integer grid $\mathbb{Z}^2$ with the y-axis pointing *down* (screen coordinates). The eight octilinear directions are numbered $k = 0, \ldots, 7$ clockwise on screen:

$$D_0 = (1,0), \quad D_1 = (1,1), \quad D_2 = (0,1), \quad D_3 = (-1,1),$$
$$D_4 = (-1,0), \quad D_5 = (-1,-1), \quad D_6 = (0,-1), \quad D_7 = (1,-1).$$

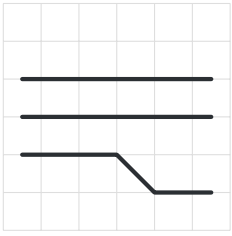


(a) The 8 directions D_k (y down).

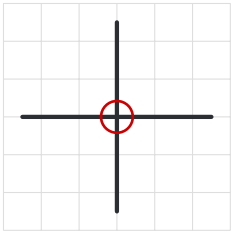


(b) Gender rule on arrival at the child v.

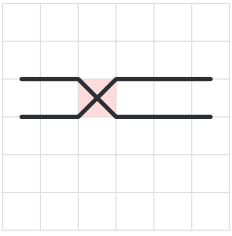
Figure 2: Directions and the gender rule. Pads are drawn with a magenta ring for female and a blue ring for male vertices (black: gender irrelevant in the illustration).



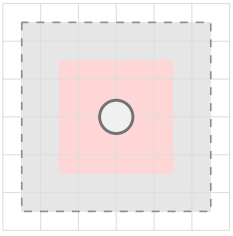
(a) Traces on adjacent grid lines: *allowed*.



(b) Shared grid point: *forbidden* by (C2).



(c) Diagonals crossing in one unit square: *forbidden* by (C3).



(d) No other pad on the red points (c=2) or grey points (c=3).

Figure 3: Legality of traces and pads. Because traces may sit on neighbouring grid lines, parallel traces pack densely; only shared grid points, crossing diagonals and pads closer than the clearance are excluded.

Direction k is *orthogonal* if k is even and *diagonal* if k is odd; turning by one index is a 45° bend. A unit step has length 1 (orthogonal) or $\sqrt{2}$ (diagonal); see Figure 2a.

Definition 1 (Drawing)**.** *A drawing assigns to every vertex v a* pad $p(v) \in \mathbb{Z}^2$ *and to every edge $e = \{u, v\}$, with u the parent of v, a* trace $\pi(e) = (q_0, q_1, \ldots, q_n)$ *of grid points with $q_0 = p(u)$, $q_n = p(v)$ and $q_i - q_{i-1} \in \{D_0, \ldots, D_7\}$ for all i. The points $q_1, \ldots, q_{n-1}$ are the* interior points *of the trace and $q_n - q_{n-1}$ is its* arrival direction.

2.3 Constraints

Let $c \in \{2, 3\}$ be the *pad clearance*. A drawing is *valid* if

(C1) **Pad clearance.** $\|p(v) - p(w)\|_\infty \geq c$ for all distinct v, w in the same tree (pads of different trees are even farther apart, Section 5).

(C2) **Exclusive interior points.** An interior point of a trace is neither a pad nor an interior point of another trace, and a trace does not visit a grid point twice.

(C3) **No crossing diagonals.** No two traces use diagonal steps inside the same unit square.

(C4) **Gender rule.** The arrival direction of the trace to v is orthogonal iff $g(v) = \mathrm{F}$ and diagonal iff $g(v) = \mathrm{M}$.

Traces of different edges may meet only at a common pad (the parent's pad). Note that (C2) does *not* forbid traces on adjacent parallel grid lines: two traces may be one grid step apart, which is what gives the drawing its board-like density (Figure 3).

2.4 Two basic properties

Proposition 1 (Planarity)**.** *A drawing satisfying (C2) and (C3) has no crossings: two traces intersect at most in a common pad.*

Proof. Every trace is a union of unit steps between grid points. Two unit steps that share no grid point can intersect only in the interior of a unit square: orthogonal steps run along grid lines and meet grid lines only at grid points, while a diagonal step meets the open interior of exactly one unit square and touches its boundary only at its two end points. Hence an intersection of two steps without a common grid point needs two diagonal steps in the same unit square, either the two crossing diagonals or the same diagonal twice; both are excluded by (C3). A common grid point of two different traces is, by (C2), never interior to either of them unless it is a pad, and a pad is shared only by traces incident to that vertex. □

Proposition 2 (Degree bound)**.** *If $c \geq 2$, a vertex of degree greater than* 8 *cannot be drawn validly.*

Proof. Consider a pad $p(v)$ and an incident trace. Its point next to $p(v)$ is at Chebyshev distance 1; it cannot be the other end pad because pads are at distance $\geq c \geq 2$ (C1), so it is an interior point. By (C2) the interior points of different traces are distinct, so the $\deg(v)$ incident traces use $\deg(v)$ distinct neighbours of $p(v)$, of which there are only eight. □

The implementation draws such vertices anyway and reports the conflicts (Section 4.7).

3 Input and validation

Adjacency list. One line per vertex, `Vertex:Neighbour,Neighbour,...`; an isolated vertex is written `D:`. A male vertex is marked with a leading $\star$ *wherever* it appears; unmarked vertices are female. Example (two trees and an isolated vertex):

```
*A:B,*C B:*A *C:*A D:*E,F *E:D F:D G:
```

Adjacency matrix. Row i is vertex $\ell(i)$ with labels $\mathrm{A},\ldots,\mathrm{Z},\mathrm{A1},\ldots,\mathrm{Z1},\mathrm{A2},\ldots$; a $\star$ at the start of a row makes that vertex male.

Validation. The input is rejected as a whole (and the current graph is kept) if any of the following occurs; every problem is reported with its line or row: inconsistent gender marks of the same vertex, malformed lines (not exactly one colon, empty or illegal names, a $\star$ not at the start), a vertex with two lines, a neighbour listed twice, self-loops, asymmetric adjacency (list or matrix), entries other than $0/1$, a row of wrong length, more than 300 vertices, and finally cycles. The forest check uses union–find with path halving over the edge list: an edge whose ends already have the same representative closes a cycle. The number of successful unions gives the number of trees, $|V| - \#\text{unions}$. In the visual builder an edge is refused if its ends are already connected (breadth-first search), so the graph stays a forest at all times.

4 Routing a large tree

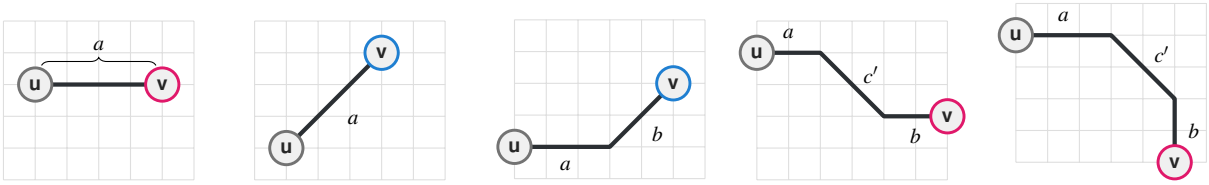
Large and medium trees are routed; this section describes the routing of one such tree.

4.1 Root and placement order

Root. If the user names a root in the tree it is used; otherwise the *centre* of the tree is used: a breadth-first search from the first vertex of the component finds an end a of a longest path, a second search from a finds the other end b, and the middle vertex of the a–b path is chosen (the classical tree centre, cf. Jordan [5]). (Small trees are not routed, Section 5.) Starting in the centre minimises the depth of the rooted tree, so the board grows evenly in all directions.

Algorithm 1: LAYOUTTREE(T, r, S) — greedy routing of one tree

```
root T at r by BFS; compute s(v); sort every ch(v) by s descending, then label
order ← BFS order of T if S.order = bfs else DFS preorder
p(r) ← (0,0);   pend(r) ← κ(r);   B ← {(0,0)}
foreach u ∈ order do
    foreach v ∈ ch(u) do
        ROUTE(u, v)                                          ▷ Algorithm 3
return pads, traces, B, list of forced routes
```



(a) Straight, female. **(b)** Straight, male. **(c)** One bend, male. **(d)** Dogleg, female. **(e)** Chamfered corner.

Figure 4: The candidate families. Segment lengths are independent; the parity of the last segment decides which gender the trace can serve.

Order. The tree is rooted by breadth-first search, sub-tree sizes $s(v)$ are computed bottom-up, and the children of every vertex are sorted by decreasing $s(v)$ (ties by label), so large sub-trees get the best room. The vertices u are then processed in one of two orders, and for each u all its children are routed one after another:

- **BFS order** (default, "generation by generation"): like the metro map, one generation is finished before the next starts, which yields a radial, balanced board.
- **DFS preorder** ("sub-tree by sub-tree"): a sub-tree is finished before its siblings' sub-trees, which clusters families together.

The root's pad is placed at $(0,0)$. Throughout the routing the algorithm maintains the occupied pads, the interior trace points, the used diagonal slots (unit square $\times$ diagonal type), the arrival direction $\delta_{\text{in}}(v)$ of each placed vertex, the bounding box B of everything drawn, and $\text{pend}(w)$, the number of children of a placed vertex w that still need a trace.

4.2 Candidate traces

For the child v of u, the first stage enumerates four families of traces starting at $P = p(u)$ (Figure 4); L is the *maximum segment length* (default 6):

(a) **Straight**: a steps in direction d_1, $1 \le a \le L$. Arrival direction d_1.
(b) **One bend**: a steps in d_1, then b steps in $d_2 = d_1 + s$, $s = \pm 1$, $1 \le a, b \le L$. Arrival direction d_2.
(c) **Dogleg (jog)**: a steps in d_1, c' steps in $d_2 = d_1 + s$ with $1 \le c' \le 3$, and b steps in d_1 again, $1 \le a, b \le L$. Arrival direction d_1. This is the PCB offset: a short sideways shift between two parallel runs.
(d) **Chamfered corner**: a steps in d_1, c' steps in $d_2 = d_1 + s$ with $1 \le c' \le 3$, and b steps in $d_3 = d_1 + 2s$. The trace turns by 90° in two 45° bends, the way real boards turn a corner (no sharp 90° corners). Arrival direction d_3.

All segment lengths are chosen independently, unlike the equal spacing of the metro map. The gender rule (C4) is applied as a filter on the arrival direction: a female child needs an even arrival direction, a male child an odd one. Since $d_1 \pm 1$ has the opposite parity of d_1, a single bend always switches between orthogonal and diagonal, so for every child both the families (a), (c), (d) (last segment with the parity of d_1) and family (b) remain available. Only 45° bends are generated in this stage; a 90° turn without chamfer appears only in the maze router, at a high cost (Section 4.5). Families (c) and (d) can be switched off together ("two-bend traces").

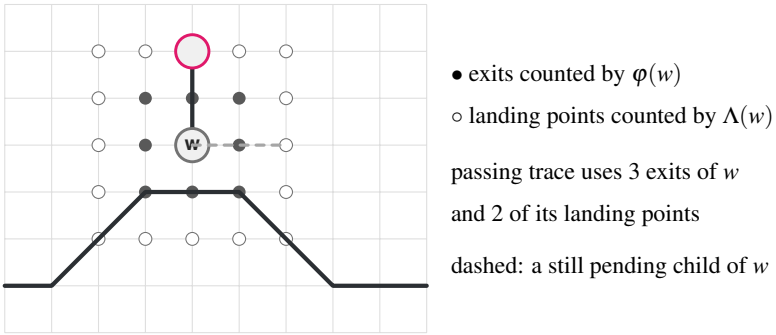


Figure 5: Reservation around a pad w with pending children. A candidate trace passing near w is accepted only if the remaining exits $\varphi(w)$ and landing points $\Lambda(w)$ stay at least need(w).

Incremental enumeration. The candidates are generated by a depth-first walk that extends the current path one unit step at a time (`walk`/`stepTo`) and undoes the step afterwards (`unstep`). A step that is illegal (Section 4.3) ends the current segment, so all longer candidates sharing that prefix are pruned immediately. Each complete candidate is passed to the evaluation (Section 4.4); a branch-and-bound test (Section 7) stops a walk as soon as no extension can beat the best candidate found so far.

4.3 Legality and the reservation rule

A unit step to a point q in direction k is *free* if q is not a pad, not an interior trace point, not already on the current candidate, and—if k is odd—neither diagonal slot of the unit square crossed by the step is in use. A candidate ending in e is accepted for evaluation if, in addition, e is not an interior trace point and no pad lies within Chebyshev distance $c-1$ of e (this includes $p(u)$, so the child is at least c away from its parent). These rules enforce (C1)–(C4) for every committed trace.

Free directions and landing points. For a placed pad w let

$$\varphi(w) = \#\{k : p(w) + D_k \text{ is free and, for odd } k, \text{ the crossing diagonal slot is unused}\},$$
$$\Lambda(w) = \#\{q : \|q - p(w)\|_\infty = 2,\ q \text{ free},\ \text{no pad other than } w \text{ within distance } c-1 \text{ of } q\}.$$

$\varphi(w)$ counts the exits still available at w (Proposition 2 shows each child needs one); $\Lambda(w)$ counts nearby points where a child pad of w could still land.

Reservation. Greedy routing has a typical failure: a trace for one vertex wanders past another pad w and takes the last exits that w needs for its own, not yet routed children. To prevent this, define need$(w) =$ pend(w) for $w \neq u$ and need$(u) =$ pend$(u) - 1$ (the trace being routed is one of u's pending children). While a candidate is extended, every new point q is charged to the waiting pads around it (Figure 5):

- a pad w with $\|q - p(w)\|_\infty = 1$ and need$(w) > 0$ receives one *hit*; the step is refused if $\varphi(w) -$ hits$(w) <$ need(w) afterwards;
- a pad $w \neq u$ with $\|q - p(w)\|_\infty = 2$ and need$(w) > 0$ receives one *landing hit*; the step is refused if $\Lambda(w) -$ landHits$(w) <$ need(w) afterwards.

φ and Λ are evaluated once per child on the committed state (cached), and hits are counted per candidate and undone with the step. The rule is conservative (a point may be counted for several pads), but it is cheap and removes most dead ends of the greedy order. The ring test is skipped for u itself because the new child pad usually lands at distance 2 from its parent.



Figure 6: Bus bonus. The trace from u to v follows the committed trace one grid line below it, including the diagonal run, so the two turn together; every highlighted step lowers its cost by σ.

4.4 Cost function

Every legal candidate π for the child v of u, ending in e, with first direction d_1 and $b(\pi)$ bends, receives the cost

$$\begin{aligned} C(\pi) \;=\; & \ell(\pi) \;-\; \sigma\,\mathrm{bus}(\pi) \;+\; 0.7\,b(\pi) \;+\; \lambda\,\Delta H(\pi) \;+\; 60\,\big[\kappa(v)-\varphi'(e)\big]^{+} \;+\; \mu\,\big[W(v)-\rho(e)\big]^{+} \\ & -\; \mathbf{1}\big[d_1=\delta_{\mathrm{in}}(u)\big] \;-\; \omega\big(\|e\|_\infty-\|p(u)\|_\infty\big) \;+\; 400\,\chi(\pi), \end{aligned} \tag{1}$$

where $[x]^{+}=\max(0,x)$ and the cheapest candidate wins (ties go to the first one enumerated). The terms are:

Length $\ell(\pi)$: number of orthogonal steps plus $\sqrt{2}$ times the number of diagonal steps.

Bus bonus $\mathrm{bus}(\pi)$ is the number of steps of π that run parallel to an already committed trace on a neighbouring grid line (Figure 6); the weight σ is the user's *bus bundling* (default 0.5). For every grid point the implementation keeps, per orientation $o=k \bmod 4$ ("–", "\", "|", "/"), the number of committed steps of that orientation touching the point. A step $q\to q'$ of orientation o counts if, for one of the two neighbouring parallel lines (offset $(0,\pm1)$ for $o=0$, offset $(\pm1,0)$ otherwise), both q and q' shifted onto that line touch a committed step of orientation o. Traces are thereby pulled into dense parallel bundles that turn together, like the buses of a circuit board.

Bends 0.7 per 45° bend; a two-bend trace costs 1.4, so it is chosen only when it pays off.

Compactness ΔH is the growth of the half perimeter $(x_{\max}-x_{\min})+(y_{\max}-y_{\min})$ of the bounding box B if the candidate is added; the corner points of the trace suffice to compute it. The weight λ is the user's *compactness* (default 1.5).

Exit look-ahead $\varphi'(e)$ is the number of free directions at the new pad, counting points of the candidate itself as blocked. If v has more children than exits, each missing exit costs 60.

Room look-ahead Only for $s(v)>1$. With radius $R(v)=\min\big(R_{\max},\max(2,\lceil 0.9\sqrt{s(v)}\,\rceil)\big)$, $\rho(e)$ counts free grid points in the $(2R+1)\times(2R+1)$ window around e, and the wanted room is $W(v)=\min\big(\beta\,(s(v)-1),\ 0.85\,((2R+1)^2-1)\big)$. Defaults: $\beta=1.5$ free points per descendant, $\mu=1$, $R_{\max}=2$. This steers big sub-trees towards open space.

Line continuation A bonus of 1 if the trace leaves u in the direction in which u itself was reached, as in the metro map, where a line runs straight through a station.

Radial growth A reward of ω per step of Chebyshev distance gained away from the root (at $(0,0)$). It is 0 by default and 0.6 in the "radial" variant of the multi-start (Section 4.8).

Collisions $\chi(\pi)$ is the number of conflicts the candidate accepts; it is non-zero only in the forced mode of Section 4.7.

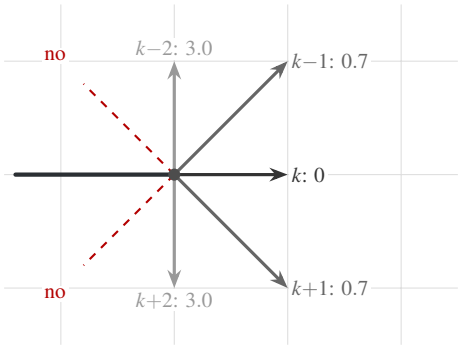


Figure 7: Moves of the maze router from a state reached in direction k (here $k = 0$) with their turn penalties (added to the step length). Turns of 135° and 180° are not allowed; unchamfered 90° turns are expensive.

Algorithm 2: FINDROUTE$(u, v, allowConflicts)$

foreach $L' \in (L, 2L)$ **do**
  enumerate straight, one-bend and (only if $L' = L$ and enabled) dogleg and chamfered-corner candidates with segments $\leq L'$, keeping the cheapest legal one by (1) (with branch and bound)
  if *found* **then return** *it*
$\pi \leftarrow$ MAZEROUTE(strict) **or** MAZEROUTE(non-strict); **if** π **then return** π
if *not* *allowConflicts* **then return** *none*
enumerate straight and one-bend candidates with segments $\leq \max(2L, w_B + h_B + 4)$ in *forced mode* (conflicts with traces allowed at cost 400 each, pads never crossed, no reservation)
if *found* **then return** *it*
return *straight trace from* $p(u)$ *to* $(x_{\max}(B) + c + 1,\ y(p(u)))$ ▷ last resort

4.5 Escalation: longer segments and the maze router

The procedure FINDROUTE (Algorithm 2) escalates when no candidate is legal: first the enumeration with L and two-bend traces (if enabled), then with $2L$ without them, then a maze router.

Maze router. The maze router is a Lee/Dijkstra search [6, 4] on states (q, k) = (grid point, direction of arrival at q), so that the cost of a turn is known. From (q, k) the next direction may be k (cost 0), $k \pm 1$ (45°, cost 0.7) or $k \pm 2$ (90°, cost 3.0, deliberately expensive because a board chamfers its corners); sharper turns are not allowed (Figure 7). A move costs its step length plus the turn cost plus a soft reservation cost of 2 for every waiting pad at distance 1 and 1 for every waiting pad $w \neq u$ at distance 2, minus the bus bonus of the step (capped so that every move still costs at least 0.2, which keeps Dijkstra's algorithm correct); the hard reservation test of Section 4.3 applies with a single hit. The goal test accepts the first popped state whose arrival direction has the right parity and whose point respects the clearance; in the *strict* run it must also offer at least $\kappa(v)$ free directions. A strict run is tried first, then a non-strict one, each limited to 60 000 expansions.

4.6 Rip-up and reroute

If FINDROUTE fails, the space around u is usually blocked by traces that were routed earlier but could have gone elsewhere. As in PCB routers [7], such traces are removed and rerouted (Algorithm 3). A committed trace to a vertex x is *movable* if $x \neq u$ and none of x's children is placed yet (so moving x drags nothing along) and the trace comes within Chebyshev distance 4 of $p(u)$. Movable traces are sorted by that distance. For $m = 1, \ldots, \min(6, \#\text{movable})$ the state is saved, the m nearest traces are ripped up (pads removed, pend of their parents increased), v is routed, and then the ripped vertices are routed again in their original order. If every one of them succeeds the result is kept; otherwise the saved state is restored and m is increased.

Algorithm 3: ROUTE(u, v) with rip-up and reroute

```
if π ← FINDROUTE(u, v, false) then commit π; return
M ← movable traces within distance 4 of p(u), nearest first
for m ← 1 to min(6, |M|) do
    save state; rip up the first m traces of M
    if π ← FINDROUTE(u, v, false) then
        commit π
        if every ripped vertex x (in order) gets a route π_x by FINDROUTE(par(x), x, false), committed
          at once then return
    restore state
commit FINDROUTE(u, v, true) and record u → v as forced
```

4.7 Forced routes

Only when everything above fails is a conflicting trace accepted: the forced mode of Algorithm 2 allows a trace to overlap other traces or cross their diagonals at a penalty of 400 per conflict (it never passes through a pad and the reservation rule is off). If even that finds nothing, a straight horizontal trace to a new pad just right of the bounding box is drawn. Every forced route is listed in the status line and its conflicts are found and marked by the verification. Forced routes are the expected outcome for vertices of degree above eight (Proposition 2) and can occur in very crowded regions of large trees (Section 8).

4.8 Multi-start

Greedy routing depends on the root and the weights. LAYOUTTREEBEST therefore runs LAYOUTTREE (without the squeeze pass) with a list of (root, settings) attempts and keeps the result with the fewest forced routes, breaking ties by the board it needs at the chosen aspect ratio α: with bounding box $w \times h$, the area of the smallest α-box around it, $\max(w, \alpha h)^2/\alpha$ (plain area would favour long thin strips). While routes conflict, attempts continue for up to 4 seconds. Once an attempt is conflict-free, further attempts for a smaller board run only within the budget T_{choose}, and only if they are expected to finish within it (judged by the duration of the previous attempt). By default $T_{\text{choose}} = 0$: in the target forests the board size is set by the packing of all trees, and extra attempts showed no gain. The roots are the chosen root and, if the user did not fix it and the tree has more than two vertices, the vertex of highest degree, the first vertex of the component, and the neighbours of the centre. The attempts are, in this order:

1. for every root: the user settings S, then the *radial* variant ($\lambda/2$, $\omega = 0.6$);
2. for every root: the *roomy* variant ($\lambda/2$, $\beta = 3$, $R_{\max} = 4$);
3. for every variant of ($\sigma = 0$), ($\lambda/2$), ($\lambda = 0.25$), ($L+3$ capped at 12 with $\lambda/2$), (the other placement order), for every root.

The squeeze pass of the next section then runs on the chosen attempt itself; no attempt is laid out twice.

4.9 Squeeze (rip-up compaction)

Greedy routing leaves slack at the border of a tree: a single vertex or trace can hold a whole row or column of the bounding box. The squeeze pass removes such slack in the manner of layout compaction in VLSI and PCB design. It repeatedly takes one side of the bounding box B (right, left, bottom, top in turn), collects every vertex whose pad or incoming trace touches that side, and rips up the sub-trees rooted at these vertices. It then routes them again, in the original order and with the normal router including rip-up, inside B shrunk by one grid step on that side (Figure 8). A shrink is kept only if no route had to be forced; otherwise the saved state is restored. The pass ends after four consecutive failures, or when the next step would exceed the time budget T_{squeeze} (default 800 ms, shared by all routed trees in proportion to their size; a step is started only if the previous one suggests it fits). A side is skipped if the root touches it, if the box is already narrower than 2 steps in that direction, or if more than $\max(12, 0.3n)$ vertices would have to be ripped up.

Algorithm 4: SQUEEZE(T_{squeeze})

```
fails ← 0;   side ← right
while fails < 4 and the next step is expected to fit into T_squeeze do
    X ← vertices whose pad or incoming trace touches side of B
    if root ∈ X or |sub-trees of X| > max(12, 0.3n) then
        fails ← fails + 1
    else
        save state; rip up the sub-trees of X (deepest first); limit ← B shrunk by 1 on side
        route the ripped vertices again in the original order, inside limit
        if a route was forced then restore state; fails ← fails + 1
        else fails ← 0
    side ← next side (right, left, bottom, top)
```

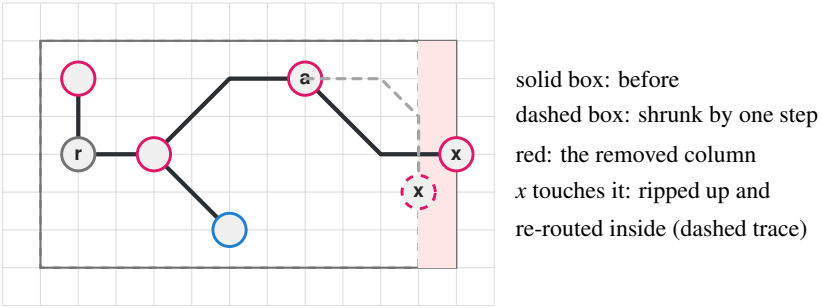


Figure 8: One squeeze step on the right side. The sub-tree of x is the only one touching the right column; it is routed again inside the box shrunk by one column, here with a chamfered corner.

Proposition 3. *The squeeze never enlarges the bounding box, never adds a forced route, and keeps every drawing valid.*

Proof. During re-routing every point outside the shrunk box is treated as blocked, so all new pads and traces lie inside it. Everything that is not ripped up does not touch the removed side, by the choice of the ripped vertices, so the box shrinks by one. All new traces are produced by the normal router and obey the same legality rules (Section 4.3); if any of them had to be forced, the complete previous state is restored. □

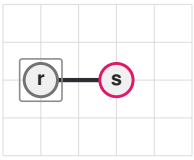
5 Forest packing

5.1 Tree classes

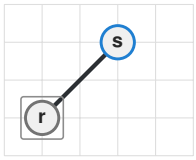
The trees of the forest are split by their number of vertices n_T into *large* ($n_T \geq 9$), *medium* ($4 \leq n_T \leq 8$) and *small* ($n_T \leq 3$) trees. Large and medium trees are routed as in Section 4; the time budgets of the squeeze and of the extra attempts are shared by these trees only, in proportion to their size. Small trees are not routed at all: their drawings come from templates (Section 5.2) and cost $O(1)$ time each.

5.2 Straight templates for small trees

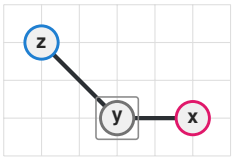
A small tree is an isolated vertex, a single edge, or a path x–y–z. Its *variants* are all drawings in which every edge is one straight trace: from the parent the trace runs in a direction whose parity matches the gender of the child (orthogonal into a female, diagonal into a male), with length c, or, for a single edge only, also $c+1$. Every vertex may be the root (or only the user's root node, if it lies in this tree). Each variant has a *rank*, its total extra length (0 for the shortest traces). Variants with the same geometry (labels ignored, after translation) are merged, keeping the lowest rank; a single edge has 4 variants, a path of three vertices at most 48 (Figure 9).



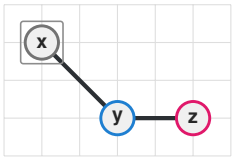
(a) Edge, female child: one straight trace.



(b) Edge, male child: one straight diagonal.



(c) Path rooted at its middle.



(d) Path rooted at an end.

Figure 9: Straight-only templates of small trees (framed: root). Each edge is one straight trace of length $c = 2$ whose direction follows the gender of the child; no template contains a bend. A single edge also has variants of length $c+1$ (rank 1), used only if no shortest one fits.

Proposition 4. *Every small tree has a valid drawing with only straight traces of length c that obeys the gender rule, for any choice of root. In particular a tree of two vertices is always drawn as one straight trace without bends.*

Proof. For a single edge $r \to s$ choose any direction D_k whose parity matches $g(s)$ and put $p(s) = p(r) + cD_k$; the two pads are at distance c and the trace has no other trace to meet. For a path rooted at its middle y, choose directions $D_a \neq D_b$ with the parities required by the two ends; the rays meet only in y, and since D_a and D_b differ by at least 1 in some coordinate, the end pads are at Chebyshev distance at least c. For a path rooted at an end, $x \to y$ in D_a and $y \to z$ in $D_b \neq -D_a$ (four choices of D_b remain); then $p(z) - p(x) = c(D_a + D_b)$ has a coordinate of absolute value at least c because $D_a + D_b \neq 0$, the two rays share only y, and their diagonal steps at y lie in different unit squares (that would need $D_b = -D_a$). □

5.3 Rotations and reflections

Lemma 1. *Let g be one of the eight symmetries of the grid (rotations by multiples of* 90° *and reflections in the axes and diagonals). Applying g to all pads and traces of a valid drawing gives a valid drawing that obeys the gender rule.*

Proof. g is an integer orthogonal map: it maps grid points bijectively to grid points, preserves Chebyshev distances (so (C1) holds), maps unit octilinear steps to unit octilinear steps, orthogonal ones to orthogonal ones and diagonal ones to diagonal ones (so (C4) holds), keeps distinct points distinct ((C2)), and maps the two diagonals of a unit square to the two diagonals of the image square ((C3)). □

Every routed tree therefore has up to eight *variants* for free (duplicates of symmetric drawings are removed).

5.4 The board grid

Packing works on one board stored as flat typed arrays over a rectangle of grid points that grows on demand (by at least 32 steps per side, copying row by row): `occ` marks occupied points (pads and trace points), `pz` the *pad zone* (points closer than c to a placed pad), `dg` the used diagonal slots of every unit square, and `near`(q) the number of occupied points within Chebyshev distance 2 of q. A variant is stored as its *footprint*: the lists of its pads, of all its points, and of its diagonal slots. A placement at offset o is *legal* if no pad of the footprint lies in the pad zone, no point is occupied and no diagonal crosses a used one; each test is one array lookup, so a legality check costs $O(|\text{footprint}|)$. Cells outside the allocated rectangle are empty.

Proposition 5. *Every packed board satisfies (C1)–(C4).*

Proof. Inside a tree the constraints hold by construction (Sections 4 and 5.2) and are preserved by translation and, by Lemma 1, by the symmetries. Between trees, a legal placement has no pad closer than c to another tree's pad, no point in common with another tree, and no crossing diagonals, which are exactly (C1)–(C3). □

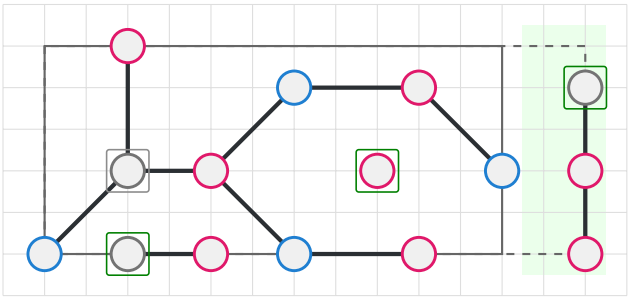


Figure 10: Nestling. Small trees (green frames: their roots) are placed into free cells inside the board first; only when none fits does the board grow, by the strip that enlarges it least.

5.5 Order and score

The largest tree (by footprint size) is placed first, in its original orientation. Then come the other large trees, then the medium trees, then the small trees by decreasing size (3, 2, 1). For a placement that turns the board box B into B' of $W \times H$ steps, the score is lexicographic, smaller first:

$$\big(f(B') = \max(W,\ \alpha H),\ \ (W+1)(H+1),\ \ \text{rank},\ \ -\text{contact}\big),$$

where α is the board aspect (width/height, default 1.6) and $\text{contact} = \sum_{q \in \text{footprint}} \min(\texttt{near}(q), 4)$ measures how snugly the variant sits against what is already drawn. Rank and contact are used only for medium and small trees (*nestling*); for large trees, ties keep the scan order.

Large trees. All variants and all offsets that keep the tree within distance c of the board are scanned; a candidate is tested for legality only if its (f, area) could beat the best one found so far.

5.6 Nestling medium and small trees

A variant can only be placed where its first pad lands on a free cell (not occupied and outside every pad zone). The search is therefore driven by free cells instead of all offsets, in two passes (Figure 10, Algorithm 5):

1. **Inside.** The free cells inside B are listed once; every variant is anchored on each of them and kept if it lies completely inside B and is legal. The board does not grow, so the best candidate has the lowest rank and, among those, the most contact. If there is one, it is taken.
2. **Grow.** Otherwise the board must grow. The boxes B pushed out by $k = 1, 2, \ldots, e + c + 1$ steps on one side (e = the largest variant extent) are sorted by (f, area). In each box only anchors in the new strip and within distance e of it are tried, and only placements that reach into the strip (all others failed in pass 1). The first box with a legal placement decides; in it, rank and then contact pick the winner.

Pass 2 makes the next small trees cheap again: they find room in the new strip during pass 1. Because the shortest variants have rank 0, a single edge uses its longer variant only when no shortest one fits anywhere in the board; in all tests every single edge was drawn at length c.

Finally the board is translated so that its box starts at $(1, 1)$, leaving a margin of one grid step.

6 Verification

After packing, the drawing is checked by a procedure that shares no data with the construction. It re-derives everything from the final coordinates:

1. **Pads**: two pads on the same point; every pair of pads closer than c (all pairs, $O(n^2)$).
2. **Traces**: start and end on the pads of the edge; every step is a unit octilinear step; no interior point is a pad; no interior point is used by two traces (overlap); no unit square contains both diagonals (crossing).
3. **Gender**: the last step of the trace to v is diagonal iff $g(v) = \text{M}$.

Algorithm 5: NESTLE(variants V) — placement of a medium or small tree

```
F_in ← free cells inside B;   best ← none
foreach v ∈ V, q ∈ F_in do
    o ← q − first pad of v
    if v + o ⊆ B and legal(v, o) then
        keep (v, o) if better by (rank, −contact)
if best then return best
foreach box B′ = B pushed out by k on one side, by increasing (f(B′), area) do
    if best and B′ is worse than best then break
    foreach v ∈ V, free anchor q in the strip of B′ (or within distance e of it) do
        o ← q − first pad of v
        if v + o ⊆ B′, v + o ⊈ B and legal(v, o) then
            keep (v, o) if better by (rank, −contact)
return best
```

Problems are listed in three reports and circled in the drawing. By the construction rules, a layout without forced routes passes all checks, and by Proposition 1 it is crossing-free. Because all coordinates are integers, the checks are exact.

7 Running time

Enumeration. Let n be the number of vertices of a routed tree and L the maximum segment length. The enumeration for one child visits at most $8L$ straight, $16L^2$ one-bend and $96L^2$ two-bend candidates (middle segment ≤ 3, dogleg or chamfered corner). Extending a path costs $O(L)$ for the self-intersection test plus $O(1)$ for the reservation and the bus bonus, so the first stage costs $O(L^3)$ per child and $O(nL^3)$ per tree. The second stage ($2L$) multiplies this by a constant. The maze router is bounded by 60 000 expansions of five moves each; rip-up repeats FINDROUTE at most $\sum_{m=1}^{6}(m+1) = 27$ times for a failing child. Multi-start and squeeze are bounded by their time budgets.

The following three speed-ups change *no* drawing (checked on random trees: identical pads and traces), but make the routing about six times faster in practice.

1. Branch and bound. Let G be the bus bonus collected so far. For a partial trace with length ℓ, b bends and box growth ΔH, the cost of every completion satisfies

$$C(\pi) \;\geq\; \underline{C} = \ell - \sigma G + 0.7\,b + \lambda\,\Delta H - 1,$$

because all look-ahead terms are non-negative and the continuation bonus is at most 1.

Proposition 6. *If $\sigma \leq 1$ and $\omega = 0$ (the defaults), $\underline{C}$ never decreases when the partial trace is extended. Hence a walk can be stopped as soon as $\underline{C} \geq C(\pi^*)$ for the best candidate π^* found so far, without changing the result.*

Proof. An extra step adds its length (1 or $\sqrt{2}$) to ℓ and at most $\sigma \leq 1$ to σG, so $\ell - \sigma G$ does not decrease; b and ΔH (the box only grows) do not decrease either. Every completion of a stopped walk therefore costs at least $C(\pi^*)$ and cannot replace π^*, which wins ties as the first one found. □

For other settings (the radial variant of the multi-start, or $\sigma > 1$) the test is switched off.

2. Reservation table. Instead of scanning the 5×5 window around every step for waiting pads (Section 4.3), the waiting pads are entered once per child into two tables, *near* (distance 1) and *ring* (distance 2, without u), keyed by grid point. A step then needs one lookup. Building the tables costs 24 entries per waiting pad.

Table 1: Random single trees (6 instances per row, averages unless stated).

n	Δ	conflict-free	points per vertex	trace length	straight	two-bend	steps in buses	forced (sum)	time [ms]
25	4	6/6	4.57	2.78	56%	8.3%	14%	0	60
50	4	6/6	5.06	2.92	50%	8.8%	22%	0	87
100	4	6/6	5.57	3.11	52%	6.4%	31%	0	649
200	4	0/6	6.47	3.67	45%	9.9%	43%	86	5268
100	6	5/6	5.71	3.38	46%	7.6%	36%	1	2006

3. Lazy look-ahead cache. The free directions and the free room around an end point depend only on the committed drawing, so they are computed once per end point and child, and only for candidates that pass the lower bound. The diagonal slots are computed without allocating objects.

Packing. A small tree has at most 48 variants with at most 5 footprint points. Pass 1 costs $O(|F_{\text{in}}| \cdot |V|)$ legality checks of constant size, where $|F_{\text{in}}|$ is the number of free cells inside the board, which shrinks as the board fills; pass 2 examines one or two strips in practice. With $|B|$ the number of board cells, packing k_s small trees costs $O(k_s\,|B|)$ in the worst case, against $O(k_s\,|B|\,|V|)$ for a scan over all offsets. Placing a large tree costs $O(8\,|B|\,|\text{footprint}|)$ in the worst case. Verification costs $O(n^2 + \sum_e |\pi(e)|)$.

8 Results

All measurements use the default settings ($L = 6$, $c = 2$, $\lambda = 1.5$, $\sigma = 0.5$, $\alpha = 1.6$, BFS order, two-bend traces and squeeze on, $T_{\text{squeeze}} = 800$ ms, $T_{\text{choose}} = 0$) and run the unchanged JavaScript of the page `electric_map.html` in Node.js 22, single-threaded. Board sizes count grid steps; "points per vertex" is the number of grid points of the board box divided by the number of vertices.

Examples. Figure 11 shows a forest of three trees (21 vertices, 18 edges) and Figure 12 a forest of the target kind with 100 vertices: three large trees, one medium tree and 21 small trees. Both are drawn here from the computed coordinates. Note the straight single edges and paths nestled around the large trees, the chamfered corners, and the parallel runs on adjacent grid lines.

Single large trees. Table 1 reports random recursive trees (vertex i attaches to a random earlier vertex whose degree is below a cap Δ) with random genders, six instances per row. Time includes the multi-start, so rows with forced routes use the 4-second retry budget.

Target forests. Table 2 compares the five test forests (100–300 vertices) laid out by the general version of the algorithm (every tree routed, packing by bounding-box footprint without symmetries or nestling) and by the specialised one described here. On a wider set of 28 random forests of the same kind (1–5 large trees, up to 90 small trees, up to 2 medium trees), the specialised version needs 4.6% less board area in total (smaller in 21 forests, larger in 2), is 5.0 times faster (20.1 s against 4.1 s in total), and draws no single edge with a bend, against 190 bent single edges before. No layout had any conflict.

Speed-ups. The three exact speed-ups of Section 7 were checked on ten random trees of 32–140 vertices with fixed roots and settings: all ten drawings are identical, and the routing time fell from 2.50 s to 0.43 s in total. The specialised packing takes 15–60 ms for forests of up to 300 vertices.

Observations. Boards need about 4.4–4.6 grid points per vertex for the target forests and 4.6–6.5 for single trees. The lower bound is about 4 with clearance 2, since a pad blocks the pad positions of its 3×3 neighbourhood. Around half of the traces are straight, and the share of trace steps in parallel runs grows with the tree size (14% at 25 vertices, 43% at 200). Trees up to 100 vertices with degree at most 4 are drawn

```
*A:B,*C,D
B:*A,*E,F,G
*C:*A,H,*I
D:*A,*J
*E:B,K,*L
F:B
G:B,*M
H:*C
*I:*C,N,*O
*J:D
K:*E
*L:*E
*M:G
N:*I
*O:*I
P:*Q,R,*S
*Q:P
R:P,*T
*S:P
*T:R
U:
```

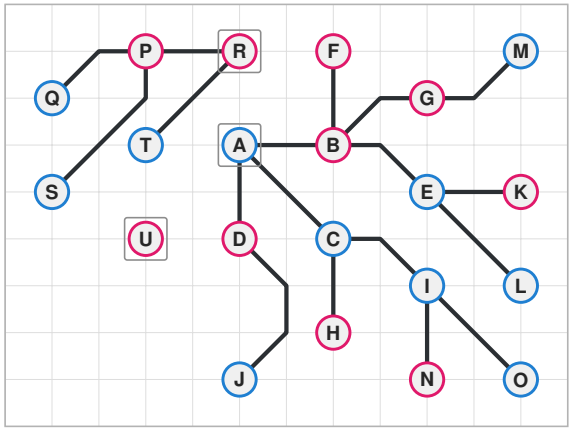


Figure 11: Input (left) and computed board (right) of a forest of three trees: 10×7 grid, 4.2 grid points per vertex, average trace length 2.50 steps, 11 of 18 traces straight, no conflicts. Framed pads are the roots; *U* is an isolated vertex.

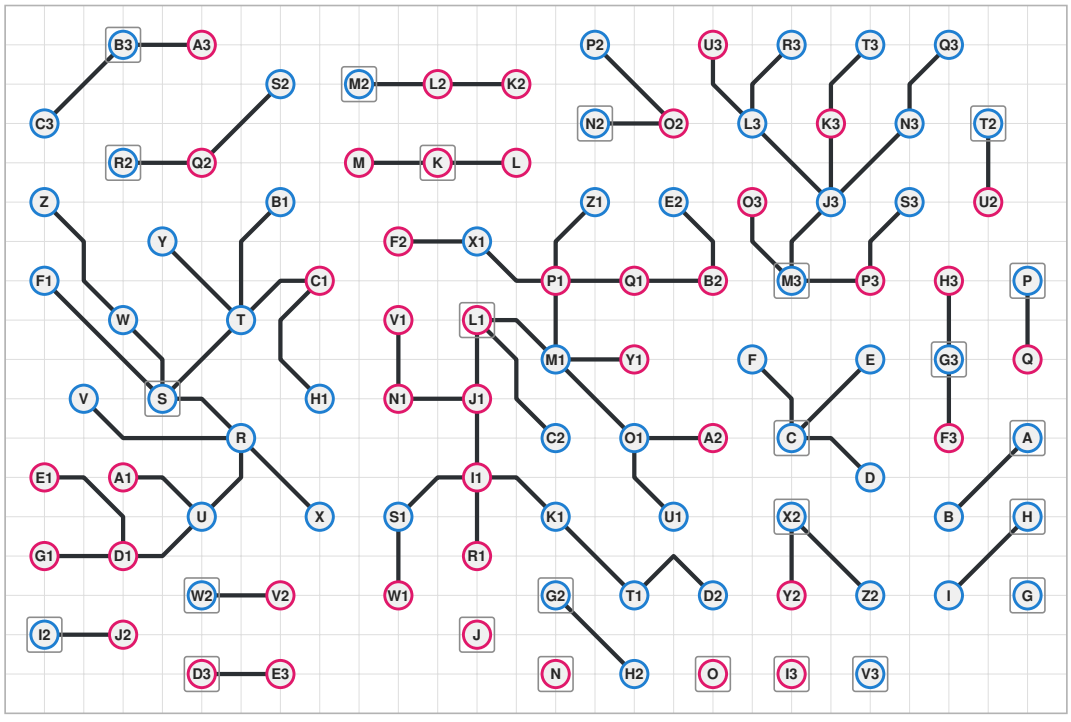


Figure 12: The test forest with 100 vertices (`forest_100.txt`: large trees of 24, 17 and 12 vertices, a medium tree of 4, and 21 small trees): 25×16 grid, 4.4 grid points per vertex, all 8 single edges straight at the shortest length, no conflicts, under 0.3 s.

Table 2: The five test forests (L/M/S = large/medium/small trees; bent = single edges drawn with a bend / all single edges).

			general version				specialised version			
n	trees	L/M/S	board	pts/vertex	bent	ms	board	pts/vertex	bent	ms
100	25	3/1/21	27×16	4.76	4/8	703	25×16	4.42	0/8	272
150	31	4/1/26	32×20	4.62	4/7	764	32×20	4.62	0/7	137
200	45	5/2/38	38×23	4.68	9/13	1176	37×23	4.56	0/13	348
250	60	5/1/54	42×26	4.64	10/18	1152	41×25	4.37	0/18	285
300	72	6/2/64	46×28	4.54	13/19	1698	45×28	4.45	0/19	381

without conflicts. At 200 vertices the greedy routing runs out of room in the dense interior and needs forced routes; a higher maximum degree has the same effect earlier.

9 Discussion and limitations

Relation to the family metro map. The Electric Board Map adopts the octilinear style, the orientation-encodes-gender rule and the generation-wise construction of [1], but it does not represent a family as one metro line with the parents at its ends. Each parent–child edge is an independent trace and the gender is read from the arrival direction at the child, so the method applies to arbitrary forests (any degree up to eight, no binary structure). Equal station spacing is replaced by compactness: segment lengths are free, which is what allows the dense board-like packing.

Specialisation. The packing is tuned for forests of large and small trees. Small trees are cheap and always straight (Proposition 4); medium trees are routed and nestled like small ones; a forest of only medium trees is still drawn correctly, but without the template shortcut.

Greedy nature. Each child is routed once and kept unless rip-up or the squeeze moves it, so early decisions can block later ones. The reservation rule, the look-ahead terms, rip-up, multi-start and the squeeze remove most such dead ends for moderate sizes but give no guarantee; Table 1 shows where the approach reaches its limit. Promising extensions are routing sub-trees bottom-up into precomputed shapes, or an exact formulation (e.g. mixed-integer programming as used for metro maps [8, 2]) for small instances.

Buses. In a compact tree every edge is only two or three grid steps long, so long buses as on a real circuit board cannot form; the bus bonus produces parallel runs mainly in larger trees.

Degree. By Proposition 2, no valid drawing exists for a vertex of degree above eight; in practice degrees close to eight already crowd the eight exits, since the parent's trace takes one of them.

Labels. Pad labels are fitted inside the pad (font size shrinks with label length). Long names, as in a real genealogy, would need larger pitches or external labels, which are outside the scope of this document.

A Parameters

Parameter	Symbol	Default	Meaning
User settings			
Root node	r	tree centre	root of its tree; other trees keep their centre
Max. segment length	L	6 (2–12)	longest segment in the first enumeration stage
Pad clearance	c	2 (or 3)	minimum Chebyshev distance between pads; length of template traces
Compactness	λ	1.5 (0–6)	weight of bounding-box growth in (1)
Bus bundling	σ	0.5 (0–1.5)	bonus per step parallel to a neighbouring trace
Board aspect	α	1.6	target width/height of the board
Squeeze time	T_{squeeze}	800 ms	budget of the squeeze pass, shared by the routed trees
Placement order		BFS	BFS (generation by generation) or DFS pre-order
Two-bend traces		on	enumerate doglegs and chamfered corners
Squeeze board		on	run the squeeze pass
Grid pitch, grid, colour, animation		18 px, off, blue, on	rendering only
Internal constants			
Tree classes		≤ 3 / 4–8 / ≥ 9	small / medium / large
Bend penalty		0.7	per 45° bend in the enumeration
Line continuation bonus		1	first direction equals arrival direction at u
Exit look-ahead		60	per missing free direction at the new pad
Room per vertex / weight	β, μ	1.5, 1	wanted free points per descendant, and its weight
Room radius	$R_{\max}$	2	cap of the room window radius
Radial reward	ω	0	per step away from the root (radial variant: 0.6)
Two-bend middle segment	c'	≤ 3	doglegs and chamfered corners
Collision penalty		400	per conflict, forced mode only
Maze turn costs		0.7 / 3.0	45° / 90° turn
Maze reservation cost		2 / 1	per waiting pad at distance 1 / 2
Maze limit		60 000	expansions per run
Rip-up		distance ≤ 4, $m \leq 6$	movable traces considered
Squeeze rip-up limit		$\max(12, 0.3n)$	vertices per squeeze step
Retry budget		4 s	per tree while routes conflict
Extra attempts	T_{choose}	0	for a smaller board (off)
Template lengths		c; $c+1$	paths; single edges also $c+1$ (rank 1)
Contact cap		4	per footprint point
Maximum vertices		300	

B Mapping to the implementation

Function	Role
`ebParseList`, `ebParseMatrix`, `ebValidateForest`	input parsing and forest validation (Sec. 3)
`ebComponents`, `ebTreeCentre`	trees and their centres (Sec. 4.1)
`ebLayoutTree`	Algorithm 1; contains `findRoute` (Alg. 2), `search`/`walk` (enumeration with branch and bound), `evaluate` (cost (1)), `busGain`, `reserveHit` (reservation table), `mazeRoute`, `route`/`ripUp` (Alg. 3), `squeeze` (Alg. 4)
`ebLayoutTreeBest`	multi-start (Sec. 4.8)
`ebSmallVariants`, `ebSmallValid`	straight templates of small trees (Sec. 5.2)
`ebMakeVariant`, `ebDihedralVariants`	footprints and the eight symmetries (Lemma 1)
`EbBoard`	board grid: `legal`, `contact`, `place`, `grow`
`ebBestPlacement`	placement score and nestling (Alg. 5)
`ebLayoutForest`	tree classes, routing, order of placement
`ebVerify`	independent verification (Sec. 6)
`ebRender`, `ebComputeStats`	SVG drawing and board statistics

References

[1] J. Korst, V. Pronk and J. J. van Wijk. A visualization of family relations inspired by the London metro map. In *Proc. 13th Int. Symposium on Visual Information Communication and Interaction (VINCI 2020)*, ACM, 2020. `https://doi.org/10.1145/3430036.3430065`

[2] H. Bast, P. Brosi and S. Storandt. Metro maps on octilinear grid graphs. In *EuroVis 2020*.

[3] E. K. Burke, G. Kendall and G. Whitwell. A new placement heuristic for the orthogonal stock-cutting problem. *Operations Research* 52(4):655–671, 2004.

[4] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik* 1:269–271, 1959.

[5] C. Jordan. Sur les assemblages de lignes. *J. Reine Angew. Math.* 70:185–190, 1869.

[6] C. Y. Lee. An algorithm for path connections and its applications. *IRE Trans. Electronic Computers* EC-10(3):346–365, 1961.

[7] R. Nair. A simple yet effective technique for global wiring. *IEEE Trans. Computer-Aided Design* 6(2):165–172, 1987.

[8] M. Nöllenberg and A. Wolff. Drawing and labeling high-quality metro maps by mixed-integer programming. *IEEE Trans. Visualization and Computer Graphics* 17(5):626–641, 2011.